

Chapter 18: Resolvers

1. Overview

A **resolver** is a piece of code the Angular Router runs *before* it activates a route, whose job is to fetch (or otherwise produce) data that the destination component needs. The router waits for the resolver to complete — resolving an `Observable`, a `Promise`, or a plain value — and only then finishes the navigation, instantiates the component, and runs change detection. By the time the component's constructor/ `ngOnInit` executes, the data is already sitting in `ActivatedRoute.data`.

The pitch for resolvers is simple: instead of a component rendering an empty shell, subscribing in `ngOnInit`, and juggling a `loading` flag while a spinner spins, the **URL doesn't even change** until the data is ready. The user clicks a link, sees the old page a beat longer, then the new page appears fully populated. No skeleton screens, no flash of empty state, no "loading..." → "here's your data" jump.

That said, resolvers are a double-edged sword and one of the most debated patterns in Angular applications — modern Angular guidance (and the Angular team's own docs) increasingly nudge you toward deferred/stream-based loading in the component (`resource()`, signals, `@defer`, or simple `ngOnInit` + async pipe) for anything that isn't fast and critical, precisely because a slow or failing resolver **blocks navigation entirely** and there is no built-in way to show a "loading" state during that wait — the whole point of a resolver is that the router is mid-navigation and no new component (hence no new template) exists yet.

This chapter covers:

- Functional resolvers (`ResolveFn`, the Angular 14.2+/15+ standard) and legacy class-based `Resolve<T>`.
- How resolved data flows into `ActivatedRoute.data` and route data bindings.
- Error handling strategies and why a resolver that throws can leave navigation in a confusing state.
- The resolvers-vs-`ngOnInit` tradeoff in depth.
- Deprecation status of class-based patterns and the router's `withComponentInputBinding()` replacement pattern.

2. Core Concepts

2.1 What a Resolver Actually Is

A resolver is a function (or, historically, a class implementing an interface) registered on a `Route` object under the `resolve` key:

```
export const routes: Routes = [  
  {  
    path: 'products/:id',  
    component: ProductDetailComponent,  
    resolve: { product: productResolver },  
  },  
];
```

`resolve` is a map from an arbitrary **key** (here `product`) to a resolver. That key becomes the property name under `route.data` (i.e., `route.data.product`). You can register multiple resolvers on one route; the router runs them **in parallel** and waits for all of them before activating the route.

2.2 Functional Resolvers (`ResolveFn<T>`) — the Modern Standard

Since Angular 14.2 (and the default pattern from v15/v16 onward with the functional router APIs and standalone), a resolver is just a function matching the `ResolveFn<T>` type:

```
type ResolveFn<T> = (  
  route: ActivatedRouteSnapshot,  
  state: RouterStateSnapshot  
) => Observable<T> | Promise<T> | T;
```

It is defined with `const` and typically uses `inject()` to pull in services, since it runs within an injection context set up by the router:

```
export const productResolver: ResolveFn<Product> = (route, state) => {  
  const productService = inject(ProductService);  
  const id = route.paramMap.get('id')!;  
  return productService.getById(id);  
};
```

Advantages over class-based resolvers:

- No boilerplate class, no `@Injectable()`, no constructor injection ceremony — just `inject()`.
- Tree-shakable and trivially composable — a resolver can call another function, wrap another resolver's logic, or short-circuit early with plain conditionals.
- Naturally works with standalone applications (no need to declare/provide the resolver as a service anywhere except that it must still be `inject()`-compatible, i.e. within Angular's DI).

2.3 Class-Based Resolvers (Legacy `Resolve<T>`)

Prior to Angular 14.2, and still supported (though **deprecated** as of Angular 15/16 in favor of the functional form), a resolver was a service implementing the `Resolve<T>` interface:

```
@Injectable({ providedIn: 'root' })  
export class ProductResolver implements Resolve<Product> {  
  constructor(private productService: ProductService) {}  
  
  resolve(  
    route: ActivatedRouteSnapshot,  
    state: RouterStateSnapshot  
  ): Observable<Product> {  
    const id = route.paramMap.get('id')!;  
    return this.productService.getById(id);  
  }  
}
```

Registered the same way, but the route config passes the **class**, not a function:

```
{ path: 'products/:id', component: ProductDetailComponent, resolve: { product:
  ProductResolver } }
```

Deprecation status: The `Resolve<T>` interface and class-based resolver pattern are marked deprecated in the Angular Router as of Angular v15/v16 (alongside `CanActivate`, `CanDeactivate`, etc. guard interfaces receiving the same treatment). They still function today, but:

- Official docs and schematics generate functional resolvers by default.
- The interface itself carries a `@deprecated` JSDoc tag pointing to `ResolveFn`.
- Interview-relevant point: this deprecation is part of a broader Angular Router push away from class-based, interface-implementing route configuration toward plain functions that use `inject()` — the same philosophical shift seen in guards (`CanActivateFn` replacing `CanActivate`) and in the DI system generally (`inject()` over constructor injection).

Migration is mechanical: turn the class's `resolve()` method into a standalone arrow function, replace `this.service` with `inject(Service)` calls, and reference the function directly in the route config instead of the class token.

2.4 Consuming Resolved Data

There are two idiomatic ways to read what a resolver produced:

1. Via `ActivatedRoute.data` (Observable), inside the component:

```
export class ProductDetailComponent implements OnInit {
  private route = inject(ActivatedRoute);
  product$ = this.route.data.pipe(map(d => d['product'] as Product));

  ngOnInit(): void {
    this.route.data.subscribe(data => {
      console.log(data['product']);
    });
  }
}
```

`ActivatedRoute.data` is an `Observable<Data>` merging **static** route data (the `data: {...}` you hardcode on the route) with resolved data (added by key). It re-emits whenever the resolvers re-run (e.g., on param changes for a reused route — see §5).

2. Via **route-data component-input binding** (`withComponentInputBinding()`) — the modern replacement pattern that removes the need to touch `ActivatedRoute` at all:

```
provideRouter(routes, withComponentInputBinding());
```

```
export class ProductDetailComponent {
  @Input() product!: Product; // auto-bound from resolve: { product: ... }
}
```

When `withComponentInputBinding()` is enabled, the router automatically sets `@Input()` properties on the routed component whose names match route **params**, **query params**, and **resolved data keys**. This is now the recommended way to consume resolver output — it eliminates the `ActivatedRoute.data` subscription boilerplate entirely and makes the component trivially testable (just set an `@Input`, no `ActivatedRoute` mock needed).

2.5 Resolvers vs. Loading Data in `ngOnInit` — The Core Tradeoff

This is the single most interview-tested conceptual question in this chapter. A side-by-side:

Dimension	Resolver	<code>ngOnInit</code> (component-level fetch)
When data arrives	Before the route activates / component is created	After the component is created, asynchronously
UX during fetch	Old view stays on screen (or default router behavior — no visual affordance unless you build one)	New view renders immediately, typically with a loading spinner/skeleton
Perceived responsiveness	Feels "stuck" if fetch is slow — no feedback that a click registered	Feels instantly responsive — immediate visual feedback, even if content isn't ready
Component simplicity	Component receives data ready-made; no <code>loading</code> / <code>error</code> state to manage internally	Component must manage <code>loading</code> , <code>error</code> , and <code>data</code> states itself (or use <code>async pipe</code> / <code>resource()</code>)
Testability	Must mock resolver behavior at the routing layer for e2e; unit testing the resolver function itself is easy (it's a pure function)	Component tests must mock the service directly; straightforward with <code>TestBed</code>
Failure handling	A rejected/errored resolver cancels navigation — user stays on the previous page, often with no visible error unless you explicitly navigate to an error route	Component can render an inline error message or retry button in place, without leaving the route
Reusability across routes	Resolver logic is shared cleanly across any route that needs the same prefetch	Fetch logic can be duplicated or needs a shared service/hook anyway
Coupling to router lifecycle	Tightly coupled — resolvers only run on navigation, not on manual re-fetch/refresh-in-place	Decoupled — can refetch anytime (button click, interval, WebSocket push) without navigating
Parallel data needs	Multiple resolvers on one route run in parallel automatically	You must manually <code>forkJoin</code> / <code>combineLatest</code> in the component if you want parallelism

Dimension	Resolver	ngOnInit (component-level fetch)
SSR / first paint	Plays well with SSR since data is ready before render — no client-side "hydration flash"	Can cause hydration mismatches or a visible loading state during SSR if not handled carefully
Modern Angular guidance	Still valid, but Angular team increasingly recommends deferring to component-level patterns (<code>resource()</code> , signals + <code>httpResource()</code> , <code>@defer</code> blocks) for non-trivial or slow fetches	Preferred default in recent Angular idioms, especially combined with <code>@defer</code> for lazy sections and skeleton UI

When resolvers genuinely win:

- The fetch is fast and reliable (e.g., a cached lookup, a small config object) — the "no flash of empty content" benefit outweighs the blocked-navigation risk.
- The component **cannot reasonably render anything meaningful** without the data (e.g., an edit form that needs the entity to populate `FormGroup` values — building a form around `undefined` and patching it later is often messier than waiting).
- You need route guards and resolvers to cooperate (e.g., a guard checks permissions, a resolver fetches the entity guards depend on) — sharing the pre-navigation phase keeps related pre-fetch logic together.
- SSR scenarios where you want the server to render fully-populated HTML without a second client-side round trip.

When component-level (`ngOnInit` /signals) loading wins:

- The fetch is slow, flaky, or hits third-party/external APIs — you want the shell to render immediately with a spinner rather than leaving the user staring at the old page.
- You want fine-grained loading states per section of a page (some data resolved, some streamed later) — resolvers are all-or-nothing per route activation.
- You want to support pull-to-refresh / manual refetch without a full navigation.
- You're using `@defer` blocks, which already give you declarative, template-level loading/error/placeholder states without any router involvement.

2.6 Static Data vs. Resolved Data

Route `data` can be static (no async involved) or resolved (async). Both live under the same `route.data` object:

```
{
  path: 'admin',
  component: AdminComponent,
  data: { title: 'Admin Panel', roles: ['ADMIN'] }, // static, available immediately
  resolve: { stats: adminStatsResolver }           // resolved, available after fetch
}
```

`route.data` in the component ends up as `{ title: 'Admin Panel', roles: ['ADMIN'], stats: {...} }` — static and resolved values are merged into one object, keyed by whatever names you chose.

2.7 Resolvers and Guards Interplay

Guards (`canActivate`, `canMatch`) run **before** resolvers. If a guard returns `false` or a `UrlTree` (redirect), resolvers for that route never execute — you don't pay the network cost for data the user isn't allowed to see. This ordering (`canMatch` → `canActivate` → `resolve` → `activate`) is a common interview trip-up: people assume resolvers run first because they "prepare" the page, but authorization is checked first.

3. Code Examples

3.1 Functional Resolver with Error Redirect

```
// product.resolver.ts
import { inject } from '@angular/core';
import { ResolveFn, Router } from '@angular/router';
import { catchError, of, EMPTY } from 'rxjs';
import { ProductService } from './product.service';
import { Product } from './product.model';

export const productResolver: ResolveFn<Product | null> = (route, state) => {
  const productService = inject(ProductService);
  const router = inject(Router);
  const id = route.paramMap.get('id');

  if (!id) {
    router.navigate(['/products']); // no id in URL -> bail out to a safe route
    return of(null);
  }

  return productService.getById(id).pipe(
    catchError((err) => {
      console.error('Failed to resolve product', err);
      // Redirect to an error page WITHOUT letting the error propagate to the router,
      // which would otherwise cancel navigation and leave the user on the old page
      // with no visible feedback.
      router.navigate(['/products', 'not-found'], {
        state: { attemptedId: id },
      });
      return EMPTY; // EMPTY completes without emitting -> navigation to /products/:id
                    // is cancelled cleanly since we've already redirected elsewhere
    })
  );
};
```

```
// app.routes.ts
import { Routes } from '@angular/router';
import { productResolver } from './product.resolver';

export const routes: Routes = [
  {
    path: 'products/:id',
```

```

loadComponent: () =>
  import('./product-detail.component').then((m) => m.ProductDetailComponent),
resolve: { product: productResolver },
},
{
  path: 'products/not-found',
  loadComponent: () =>
    import('./product-not-found.component').then((m) => m.ProductNotFoundComponent),
},
];

```

Key detail: returning `EMPTY` (or any observable that completes without emitting) tells the router "this navigation is done, but there's nothing to resolve" — combined with an imperative `router.navigate()` inside `catchError`, this is the standard pattern for redirecting on resolver failure instead of letting the error bubble up and silently cancel the navigation.

3.2 Consuming Resolved Data via `ActivatedRoute.data`

```

// product-detail.component.ts
import { Component, inject } from '@angular/core';
import { ActivatedRoute } from '@angular/router';
import { AsyncPipe } from '@angular/common';
import { map } from 'rxjs';
import { Product } from './product.model';

@Component({
  selector: 'app-product-detail',
  standalone: true,
  imports: [AsyncPipe],
  template: `
    @if (product$ | async; as product) {
      <h1>{{ product.name }}</h1>
      <p>{{ product.description }}</p>
      <strong>{{ product.price | currency }}</strong>
    }
  `,
})
export class ProductDetailComponent {
  private route = inject(ActivatedRoute);

  product$ = this.route.data.pipe(map((data) => data['product'] as Product));
}

```

Snapshot-based alternative (fine when the component is never reused across a param-only navigation, i.e., the route always re-creates the component):

```

export class ProductDetailComponent {
  private route = inject(ActivatedRoute);
  product = this.route.snapshot.data['product'] as Product;
}

```

3.3 Consuming Resolved Data via Route-Data Component-Input Binding

```
// app.config.ts
import { ApplicationConfig } from '@angular/core';
import { provideRouter, withComponentInputBinding } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [provideRouter(routes, withComponentInputBinding())],
};
```

```
// product-detail.component.ts
import { Component, Input } from '@angular/core';
import { CurrencyPipe } from '@angular/common';
import { Product } from './product.model';

@Component({
  selector: 'app-product-detail',
  standalone: true,
  imports: [CurrencyPipe],
  template: `
    <h1>{{ product.name }}</h1>
    <p>{{ product.description }}</p>
    <strong>{{ product.price | currency }}</strong>
  `,
})
export class ProductDetailComponent {
  @Input() product!: Product; // key 'product' matches resolve: { product: productResolver }
}
```

No `ActivatedRoute` injection at all — the router sets `product` as a plain `@Input` the moment the route activates, because the resolver's key (`product`) matches the input's name. This is the pattern the Angular team now recommends: it makes the component a plain, easily-unit-testable class that just happens to receive its data as an input, indistinguishable from a component used outside the router entirely.

3.4 Legacy Class-Based Resolver (for comparison / migration reference)

```
// product.resolver.ts (legacy, deprecated pattern)
import { Injectable } from '@angular/core';
import { Resolve, ActivatedRouteSnapshot, RouterStateSnapshot, Router } from '@angular/router';
import { Observable, of } from 'rxjs';
import { catchError } from 'rxjs/operators';
import { ProductService } from './product.service';
import { Product } from './product.model';

@Injectable({ providedIn: 'root' })
export class ProductResolver implements Resolve<Product | null> {
```

```

constructor(private productService: ProductService, private router: Router) {}

resolve(
  route: ActivatedRouteSnapshot,
  state: RouterStateSnapshot
): Observable<Product | null> {
  const id = route.paramMap.get('id')!;
  return this.productService.getById(id).pipe(
    catchError(() => {
      this.router.navigate(['/products', 'not-found']);
      return of(null);
    })
  );
}

```

Migrating this to the functional form is a matter of flattening the class into an arrow function and swapping constructor injection for `inject()` calls, as shown in §3.1.

4. Internal Working

4.1 Where Resolvers Sit in the Navigation Pipeline

The Angular Router models navigation as an RxJS pipeline of sequential phases. Simplified, the router's internal navigation transitions observable runs through operators approximately in this order:

1. `NavigationStart` event fires.
2. **URL matching / recognition** — `Router` matches the URL against the route config tree, builds an `ActivatedRouteSnapshot` tree (`recognize` phase). Emits `RoutesRecognized`.
3. **canMatch guards** — considered during recognition itself, can cause the router to skip a route and try the next matching config.
4. **canActivateChild / canActivate guards** — run for the matched tree, outer to inner. Any guard returning `false` or a `UrlTree` aborts/redirects the navigation **before resolvers run**. Emits `GuardsCheckStart / GuardsCheckEnd`.
5. **Resolvers (resolve)** — the router now iterates every activated route in the new tree that declares a `resolve` map, and for each one, invokes the resolver function/class with `(route, state)`. This emits `ResolveStart`.
 - All resolvers across the whole route tree are merged into a single observable (conceptually similar to combining multiple observables and waiting for all to complete), typically via `forkJoin`-like semantics per route, so **resolvers on the same route run in parallel**, and (in modern Router implementations) resolvers across parent/child activated routes are also run together.
 - Each resolver's return value is normalized: if it returns a plain value, the router wraps it as `of(value)`; if a `Promise`, it's converted via `from()`; if an `Observable`, it's used as-is, and the router takes the **first emitted value** (conceptually akin to piping through `take(1)`) — a resolver observable that never emits will **hang navigation forever** since the router is waiting on it to produce a value before proceeding.

- The results are assigned onto `route.data` for each corresponding `ActivatedRouteSnapshot`, keyed by whatever key you registered them under. Emits `ResolveEnd`.

6. **Component instantiation** — only now does the router create/reuse the component instance for the target route via the outlet.

7. **Change detection runs**, the component's `ngOnInit` fires, `ActivatedRoute.data` observable emits.

8. `NavigationEnd` (or `NavigationCancel` / `NavigationError` if something failed along the way).

The crucial mental model: **the URL bar and browser history do not update, and the previous component stays mounted, until step 5 fully completes.** This is precisely why a slow resolver produces a "did my click even register?" UX problem — nothing on screen changes until the whole chain finishes.

4.2 How Resolved Data Attaches to the Snapshot

Each node in the route tree is an `ActivatedRouteSnapshot`, which has a mutable-internally `data` property. During the resolve phase, the router's `resolveData` operator walks the new snapshot tree, runs each route's resolvers, and merges the resulting key/value pairs into that snapshot's `data` object alongside any static `data` you configured. The `ActivatedRoute` (the long-lived, non-snapshot object your component injects) exposes this as:

- `route.snapshot.data` — a plain object, current value only.
- `route.data` — an `Observable<Data>` that emits a new object every time the router updates that route's snapshot (initial activation, and again on **any later re-resolve**, e.g. from a param change on a reused route — see §5.3).

Internally, `ActivatedRoute.data` is backed by a `BehaviorSubject`-like mechanism fed by the router's navigation transitions; that's why subscribing to it in `ngOnInit` reliably gets the current value immediately (no missed emission), and why it re-emits without you resubscribing when the router re-resolves the same reused route instance.

4.3 Parallel Execution and `forkJoin` Semantics

If a single route declares:

```
resolve: { product: productResolver, reviews: reviewsResolver, related: relatedResolver }
```

all three functions are invoked essentially simultaneously (not sequentially), and the router waits for **all three** to complete before activating the route — functionally equivalent to:

```
forkJoin({
  product: productResolver(route, state),
  reviews: reviewsResolver(route, state),
  related: relatedResolver(route, state),
})
```

This means the *slowest* resolver determines total navigation latency — a fast `product` resolver and a slow `related` resolver still make the user wait for `related` before seeing anything, unless you deliberately decouple `related` into a component-level fetch instead.

4.4 Dependency Injection Context Inside a Resolver

Functional resolvers run inside an Angular **injection context** established by the router specifically for that call — this is what makes `inject()` work inside a plain arrow function that isn't a class. The router uses `runInInjectionContext()` internally (conceptually) against the relevant environment injector (the route's own injector if it declares `providers`, otherwise the parent environment injector) before invoking the resolver function. This is also why you cannot call `inject()` *asynchronously* after the resolver function's synchronous body has returned — e.g., inside a `.then()` callback fired later — the injection context is only valid during the synchronous execution of the resolver function itself; grab everything you need with `inject()` up front, then use those references inside any async callbacks.

5. Edge Cases & Gotchas

5.1 Blocked Navigation UX ("Did My Click Register?")

Because the previous view stays fully interactive and the URL doesn't change until resolvers finish, a slow resolver (a flaky API, a cold serverless function, a large payload) creates a UX dead zone: the user clicked a link/button, nothing visibly happened, and they don't know whether to click again. Angular provides **no built-in loading indicator** for this phase — you must build one yourself, typically by subscribing to router events:

```
this.router.events.pipe(
  filter((e): e is NavigationStart | NavigationEnd | NavigationCancel | NavigationError =>
    e instanceof NavigationStart || e instanceof NavigationEnd ||
    e instanceof NavigationCancel || e instanceof NavigationError
  )
).subscribe(e => {
  this.isNavigating = e instanceof NavigationStart;
});
```

...and rendering a top-of-page progress bar or spinner keyed off `isNavigating`. This is a very common real-world gotcha: teams add resolvers for "clean" navigation and then have to retroactively bolt on a global loading bar to fix the perceived unresponsiveness they introduced.

5.2 Errors Causing "Stuck" Navigation

If a resolver's observable **errors** (rather than completing with a value), and you don't catch it inside the resolver, the router treats it as a failed navigation:

- The in-flight `Navigation` transition throws, emitting `NavigationError`.
- The router **stays on the current route** — the URL reverts/doesn't change, the previous component remains active.
- **No error is shown to the user by default.** Unless you have a global `router.events` subscriber logging/handling `NavigationError`, the failure is silent from the user's perspective: they clicked, waited, and nothing happened, with no console-visible feedback in production builds unless you've wired up error reporting.
- This is why virtually every real resolver should have a `catchError` that either supplies a fallback value or explicitly redirects (as in §3.1) — letting the raw error propagate is almost always the wrong choice.

A more subtle version of this: if the resolver returns `EMPTY` (or any observable that completes without ever emitting a value) **without** having triggered a redirect first, the router waits for "the first value" that never arrives — this can hang navigation indefinitely rather than erroring out, which is arguably worse than an explicit error because there's no `NavigationError` event to even detect the problem.

5.3 Resolvers Re-Running on Param Changes (and When They Don't)

If a route is configured such that the **same component instance is reused** across a navigation — the classic case being `/products/1` → `/products/2` where only the `:id` param changes and the route config node is identical — Angular's default `RouteReuseStrategy` reuses the component instance rather than destroying/recreating it. In this scenario:

- Resolvers **do re-run** on the param change (this is the default and expected behavior — the router treats resolve as part of "activating" the route data for the new param set, not tied to component construction).
- Because the component isn't recreated, `ngOnInit` does **not** fire again — this is precisely why subscribing to `route.data` (the Observable) rather than reading `route.snapshot.data` once in `ngOnInit` matters: only the Observable form picks up the second (and subsequent) resolved values on a reused component.
- If you used the snapshot (`route.snapshot.data['product']`) captured once in `ngOnInit`, navigating from `/products/1` to `/products/2` will silently leave the UI showing product 1's data even though the resolver ran again and fetched product 2 — a very common and hard-to-spot bug.

Related gotcha: if a **custom** `RouteReuseStrategy` is in play (common in tabbed/cached-route apps) that reuses routes more aggressively, resolvers may not re-run at all when you expect them to, because the router may treat the route as fully "already resolved" and skip resolution — always verify resolver re-execution behavior empirically when a custom reuse strategy is present.

5.4 Resolvers Run on Every Matching Navigation, Not Just "First Load"

A resolver isn't a one-time setup hook — it runs every single time its route is navigated to and activated, including repeated visits in the same session (e.g., navigating away and back). If the resolver always hits the network with no caching, you can end up with redundant requests on every back/forward navigation. Mitigate by caching inside the service the resolver calls (e.g., `shareReplay(1)` keyed by id, or an in-memory cache map), not inside the resolver itself.

5.5 Resolvers and Lazy-Loaded Modules/Routes

A resolver registered on a lazily-loaded route only becomes known to the router once that lazy chunk is loaded — this is usually transparent, but it means the resolver's own dependencies (via `providedIn: 'root'` services or route-level `providers`) must be resolvable from whatever injector is active at that point; a resolver depending on a service provided only in a *different* lazy module's injector will throw a `NullInjectorError` at resolve time, not at compile time, so this class of bug only surfaces at runtime navigation.

5.6 Resolvers Do Not Run on Query-Param-Only Changes by Default

Adding/changing a query param (`?sort=asc` → `?sort=desc`) on the same route **does not**, by default, cause resolvers to re-run unless the resolver itself reads `route.queryParamMap` and you've structured the route/component to actually re-navigate (as opposed to the component just reacting to `ActivatedRoute.queryParams` itself). This trips people up when they expect a resolver keyed off a query param to refresh automatically — in practice, query-param-driven data fetching is usually better handled directly in the component via `queryParamMap.subscribe(...)`, not via a resolver.

5.7 withComponentInputBinding() Silent Mismatches

When using route-data input binding (§3.3), if the resolver's key doesn't exactly match an `@Input()` name on the component (typo, casing mismatch), there is **no error** — the input simply stays `undefined`/unset, and the component silently renders with missing data. This fails silently rather than loudly, unlike a typo in an `ActivatedRoute.data` subscription's key lookup (`data['product']`), which is *also* silent (returns `undefined`) — either way, key mismatches between resolver registration and consumption are a common, hard-to-catch bug because TypeScript cannot statically check the string keys against the `@Input()` names.

6. Interview Questions & Answers

Q1. What is a resolver in the Angular Router, in one sentence?

A resolver is a function (or, historically, a class) registered on a route's `resolve` property that the router runs and waits on before activating that route, so the target component receives its data already loaded via `ActivatedRoute.data`.

Q2. What's the difference between a functional resolver and a class-based resolver?

A functional resolver is a plain function matching `ResolveFn<T>`, using `inject()` for dependencies, registered directly in the route config. A class-based resolver implements the `Resolve<T>` interface with a `resolve()` method, uses constructor injection, and is registered by passing the class itself. Functional resolvers are the modern, recommended style (Angular 14.2+); class-based resolvers are deprecated but still functional.

Q3. Where does resolved data end up, and under what key?

Under `route.data` (both the snapshot's `data` object and the `ActivatedRoute.data` Observable), keyed by whatever property name you used in the `resolve: { key: resolverFn }` map — not by the resolver's function/class name.

Q4. Can multiple resolvers be attached to one route, and do they run in sequence or parallel?

Yes — `resolve` accepts a map of multiple keys to multiple resolvers. They run in parallel (conceptually a `forkJoin`), and the router waits for all of them to complete before activating the route. Total wait time is bounded by the slowest resolver, not the sum of all of them.

Interviewer intent: This checks whether the candidate understands the router's internal wiring versus just knowing resolvers exist. A wrong answer ("they run one after another") suggests the candidate has only used resolvers superficially and hasn't reasoned about why a route with several resolvers can still feel slow even though each individual one is fast — it's whichever is slowest, run concurrently.

Q5. What happens if a resolver's observable never completes or never emits a value?

Navigation hangs indefinitely. The router is waiting for the resolver's observable to emit at least once before it will activate the route; if it never emits (e.g., an incorrectly used `EMPTY` without a redirect, or a subject that's never fed a value), the user is stuck on the current page with no error and no indication anything is wrong.

Q6. How should a resolver handle an error from its data-fetching call?

It should `catchError` internally and either (a) return a fallback value (e.g., `of(null)` or a default object) so navigation proceeds and the component handles the "no data" case, or (b) imperatively call `router.navigate()` to redirect to an error/fallback route and then return `EMPTY` (or another completing-without-value observable) so the original navigation is cleanly abandoned. Letting the error propagate unhandled causes a silent `NavigationError` with the user stuck on the previous page.

Q7. Why would you choose loading data in `ngOnInit` over using a resolver, even though resolvers give you "data ready before render"?

Because resolvers block the entire navigation on the slowest fetch, with no built-in loading UI and a router-level "stuck" feeling if the request is slow or unreliable. Component-level fetching lets the new view render immediately (with its own spinner/skeleton), supports manual refetch without navigating, and doesn't leave the user staring at the old page. It's the right choice whenever the fetch is slow, optional, or you want granular loading states.

Interviewer intent: This is the chapter's central tradeoff question. Interviewers want to see the candidate isn't reflexively pro-resolver ("resolvers are always better because the component is 'ready'") — they want an explicit acknowledgment of the blocked-navigation cost and a rule of thumb for when each approach fits.

Q8. Are class-based resolvers deprecated? What replaces them?

Yes — the `Resolve<T>` interface (along with the class-based guard interfaces like `CanActivate`) is marked deprecated as of Angular 15/16 in favor of functional resolvers using the `ResolveFn<T>` type and `inject()`. Existing class-based resolvers still work; new code and Angular schematics default to the functional form.

Q9. What is `withComponentInputBinding()` and how does it relate to resolvers?

It's a `provideRouter()` feature that automatically binds route params, query params, and resolved route data to matching `@Input()` properties on the routed component, by name. For resolvers specifically, it means a `resolve: { product: productResolver }` entry automatically populates `@Input() product` on the component — no `ActivatedRoute` injection or `.data` subscription required. It's the modern recommended pattern for consuming resolver output.

Q10. If a route reuses the same component instance across a param change (e.g., `/products/1` → `/products/2`), does the resolver re-run? Does `ngOnInit` re-run?

The resolver re-runs (resolution is tied to route activation/param changes, not component construction). `ngOnInit` does **not** re-run, because the same component instance is reused. This is why you must subscribe to the `ActivatedRoute.data` Observable (or use input binding, which updates the `@Input()` reactively) rather than reading `route.snapshot.data` once in `ngOnInit` — the snapshot approach will show stale data from the first navigation.

Interviewer intent: This probes whether the candidate actually understands route reuse semantics rather than just "resolvers run before the component." It's a realistic bug pattern (stale snapshot data on param-only navigation) that shows up in real codebases, so a candidate who's hit it in practice will answer confidently; one who hasn't will often incorrectly assume `ngOnInit` always re-fires.

Q11. Where do guards fit relative to resolvers in the navigation pipeline?

Guards (`canMatch`, then `canActivateChild` / `canActivate`) run **before** resolvers. If any guard returns `false` or a `UrlTree`, the navigation is redirected/aborted and resolvers for that route never execute. This ordering avoids paying for a data fetch the user isn't authorized to see.

Q12. Does changing only a query parameter (not a path param) cause a resolver to re-run?

Not by default. Resolvers respond to route activation and matched-route changes (including path param changes on a reused route); a query-param-only change on the same matched route typically does not re-trigger resolution unless you explicitly structure the resolver/route to react to it. Query-param-driven data is usually better handled directly in the component via `ActivatedRoute.queryParamMap`.

Q13. Can a resolver redirect the user instead of, or in addition to, providing data?

Yes. A common pattern is injecting `Router` inside the resolver and calling `router.navigate([...])` when data can't be found (e.g., 404) or a precondition fails, then returning `EMPTY` (or a fallback value) so the router treats the current navigation as complete/abandoned rather than erroring.

Q14. What's the risk of putting expensive/non-cached logic directly inside a resolver?

Since resolvers re-run on every matching navigation (including repeated back/forward visits, not just first load), a resolver with no caching will re-fetch on every visit to the route, potentially causing redundant network calls. The fix is to cache at the service layer (e.g., `shareReplay(1)` per key, an in-memory map, or HTTP cache headers), not inside the resolver function itself, since the resolver is invoked fresh each time.

Interviewer intent: Tests whether the candidate treats resolvers as "special" router magic or correctly understands they're just regular function calls invoked at a particular lifecycle point — the caching discipline needed is identical to any other repeated data-fetching code path.

Q15. Why can't you safely call `inject()` inside a `.then()` / `.subscribe()` callback within a resolver, but you can call it in the resolver's synchronous body?

Angular establishes an injection context only for the synchronous execution of the resolver function call itself (the router effectively wraps the invocation in something equivalent to `runInInjectionContext()`). Once that synchronous call returns — e.g., control returns to the microtask queue after a `.then()` schedules a callback — the injection context is no longer guaranteed to be active, so calling `inject()` there throws `NG0203` (`inject()` must be called from an injection context). The fix is to call `inject()` for everything you need up front in the resolver's synchronous body, storing references in local variables the async callback can close over.

7. Quick Revision Cheat Sheet

- **Resolver** = code the router runs and waits on **before** activating a route; result lands in `route.data[key]`.
- **Functional resolver** (modern, Angular 14.2+): `const fooResolver: ResolveFn<T> = (route, state) => {...}`, uses `inject()`.
- **Class-based resolver** (`implements Resolve<T>`): deprecated since v15/16; still works, migrate by flattening to a function.
- Register: `{ path: '...', resolve: { key: resolverFn } }` — the map **key** is what shows up in `route.data`.
- Multiple resolvers on one route run **in parallel**; navigation waits for the **slowest** one.
- Consume via `route.data` Observable (survives component reuse) or `route.snapshot.data` (one-shot,

stale on reuse) or, preferably, `withComponentInputBinding()` + matching `@Input()` name.

- Order in the pipeline: `canMatch` → `canActivateChild` → `resolve` → component created → `ngOnInit` → `NavigationEnd`.
- A resolver observable must **emit and complete**; if it never emits, navigation hangs forever with no error.
- Always `catchError` inside a resolver — either supply a fallback value or `router.navigate()` + return `EMPTY`; an uncaught error silently strands the user on the previous page (`NavigationError`, no default UI).
- No built-in loading indicator for the resolve phase — build one from `router.events` (`NavigationStart` / `End` / `Cancel` / `Error`) if fetches might be slow.
- On a **reused component** (param-only navigation), resolvers re-run but `ngOnInit` does not — use the `data` Observable, not a captured snapshot.
- Query-param-only changes do **not** re-trigger resolvers by default.
- Resolvers re-run on **every** matching navigation, not just first load — cache at the service layer if the fetch is expensive.
- `inject()` inside a resolver only works in the function's synchronous body, not inside a later `.then()` / `.subscribe()` callback.
- Rule of thumb: resolvers for **fast, critical, can't-render-without-it** data; component-level (`ngOnInit` / `signals` / `resource()` / `@defer`) loading for **slow, optional, or refreshable** data.

Created By - Durgesh Singh